MINOR PROJECT REPORT

Retail Sales Forecasting for a 45-Store Retail Chain

Weekly store-level demand forecasting with statistical and machine-learning models

Submitted by

Veer Sirohia

B.Tech in Data Science

Manipal University Jaipur

veersirohia@gmail.com · github.com/veersirohia16

October 2026

Abstract

Retailers plan staffing, stock and cash flow around how much each store will sell in the coming weeks. This project builds and compares forecasting models that predict weekly sales for each of the 45 stores in the public Walmart Store Sales dataset (421,570 department-level records, February 2010 to October 2012). After cleaning and aggregating the data to store-week level and studying it through exploratory analysis, six models were trained on two years of history and tested on the following 39 unseen weeks: two baselines (naive and seasonal naive), Holt-Winters exponential smoothing implemented from scratch, ridge regression, a random forest and a tuned gradient boosting model.

The **random forest**, using engineered seasonal features such as last year's sales scaled by each store's growth, gave the best result: a weighted mean absolute error (WMAE) of **$52K** per store-week, **12.4% lower** than the seasonal-naive baseline, with a mean absolute percentage error of **4.8%** per store and **2.6%** for the chain total. 32 of the 45 stores were forecast within 5% error. The final model was used to produce a 13-week holiday-season forecast (November 2012 to January 2013) for every store.

Contents

1. Introduction 4

2. Objectives 5

3. Dataset 6

3.1 Source 6

3.2 Data quality 6

4. Methodology 7

4.1 Tools and technologies 7

4.2 Workflow 7

4.3 Forecasting setup and train/test split 7

4.4 Feature engineering 8

4.5 Models 8

4.6 Hyper-parameter tuning 9

4.7 Evaluation metrics 9

5. Code and Implementation 10

5.1 Project structure 10

5.2 Key code 10

6. Results and Observations 15

6.1 Exploratory data analysis 15

6.2 Model comparison 18

6.3 Error analysis 19

6.4 Forecast for the next holiday season 20

7. Conclusion 22

Key learnings 22

Limitations and future work 23

8. References, Links and Submitted Files 24

Dataset and links 24

References 24

Files in the submission 24

1. Introduction

Sales forecasting is one of the most common and most valuable applications of data science in retail. A store that expects a busy week schedules more staff and orders more stock; a store that over-estimates ends up with idle staff and unsold inventory, and one that under-estimates loses sales to empty shelves. Because these decisions are made per store and per week, the forecast needs to be at that same level.

Retail sales are a good forecasting problem because they combine several patterns at once: a strong yearly season (the November-December holiday peak), holiday spikes that move by a few days each year, differences between large and small stores, slow growth or decline over time, and possible effects of outside factors such as fuel prices, temperature, inflation, unemployment and promotional markdowns.

This project works with the Walmart Store Sales dataset published on Kaggle, which contains almost three years of weekly sales for 45 stores. It follows a complete data science workflow: data quality checks and cleaning, exploratory data analysis, feature engineering, model building, honest time-based evaluation, comparison against simple baselines, interpretation of the best model, and finally a forward-looking forecast that a business could use.

2. Objectives

The project set out to achieve the following goals:

- **Understand the data:** check the quality of the raw files, clean them and combine them into one analysis-ready table at store-week level.
- **Find the drivers of sales:** use exploratory analysis to measure seasonality, holiday effects, the role of store type and size, and the influence of external factors.
- **Build several forecasting models** from different families (baseline, statistical, machine learning) and engineer features without leaking future information.
- **Evaluate them fairly** on a hold-out period that the models never saw, using MAE, RMSE, MAPE and the holiday-weighted WMAE metric.
- **Beat a strong baseline:** the seasonal-naive forecast ("same week last year") is hard to beat on seasonal data; the target was a clear, measurable improvement over it.
- **Explain and apply the best model:** identify which features drive its predictions, where it fails, and use it to forecast the next holiday season for every store.

3. Dataset

3.1 Source

The data comes from the Kaggle competition *Walmart Recruiting - Store Sales Forecasting* (kaggle.com/c/walmart-recruiting-store-sales-forecasting). The copy used here was taken from the public GitHub repository github.com/ShiviBhatt/E-Commerce_Sales_Forecasting (folder `data/`), which mirrors the original competition files. All files are included in the submission under `data/raw/`.

File	Rows	Columns	Description
train.csv	421,570	Store, Dept, Date, Weekly_Sales, IsHoliday	Weekly sales for each department of each store, Feb 2010 - Oct 2012
features.csv	8,190	Store, Date, Temperature, Fuel_Price, MarkDown1-5, CPI, Unemployment, IsHoliday	Regional weather and economic indicators and promotional markdowns per store-week
stores.csv	45	Store, Type, Size	Store format (A, B or C) and floor area in square feet

Table 1. Raw data files used in the project.

The competition also provides a `test.csv` without sales values (it was used for leaderboard scoring), so it cannot be used to measure accuracy. Instead, this project creates its own test period from the last 39 weeks of `train.csv`, for which the true sales are known.

3.2 Data quality

Check	Result
Stores / departments / weeks	45 / 81 / 143
Date range	2010-02-05 to 2012-10-26 (weekly, each week ends on a Friday)
Duplicate rows, missing values in train.csv	0, 0
Negative Weekly_Sales values	1,285 department-weeks (product returns exceeding sales)
Missing MarkDown1-5 in features.csv	50.5% to 64.3% (markdowns were only recorded from Nov 2011)
Missing CPI and Unemployment	7.1% (only in weeks after the sales data ends, May-Jul 2013)
Store-weeks after aggregation	6,435 = 45 stores x 143 weeks, no gaps, no duplicates

Table 2. Results of the data-quality checks (outputs/results/data_quality_report.csv).

Cleaning decisions. Negative department sales are genuine accounting entries (returns), so they were kept and simply summed into the store total. Missing markdown values mean either "not yet recorded" or "no markdown that week"; both were filled with zero and a separate flag *MarkDown_Recorded* keeps the information about which weeks had markdown data. Missing CPI and unemployment values only occur in future weeks and were handled by carrying the last known value forward when forecasting.

4. Methodology

4.1 Tools and technologies

Tool	Used for
Python 3	Programming language for the whole pipeline
pandas, NumPy	Loading, cleaning, merging and aggregating the data; feature engineering
scikit-learn	Ridge regression, random forest, gradient boosting, pipelines, permutation importance
SciPy	Numerical optimisation (L-BFGS-B) of the Holt-Winters smoothing parameters
Matplotlib	All 12 charts, with a shared colour-blind-safe style
Git / GitHub	Obtaining the public dataset

Table 3. Tools and libraries.

4.2 Workflow

The project is organised as a four-step pipeline; `main.py` runs all steps in order and every step can also be run on its own.

1. Data preparation	2. Exploratory analysis	3. Modelling & evaluation	4. Future forecast
Load 3 CSVs, quality checks, aggregate 421,570 rows to 6,435 store-weeks, merge features	Trend, seasonality, holidays, store type/size, external factors, departments (6 charts)	Time-based split, feature engineering, 6 models, tuning, 4 metrics, error analysis (5 charts)	Refit best model on all data, forecast Nov 2012 - Jan 2013 for all 45 stores (1 chart)

Figure 1. Project pipeline.

4.3 Forecasting setup and train/test split

Forecast target: total weekly sales of each store, i.e. 45 parallel weekly time series. The store level was chosen because that is where staffing and stock decisions are taken.

Time-based split. A random train/test split would let the model learn from weeks that come after the weeks it is tested on, which is impossible in real use. The data was therefore split by date:

Period	Dates	Weeks	Store-weeks	Purpose
Training	5 Feb 2010 - 27 Jan 2012	104	4,680	Fit all models (two full years)
of which validation	4 Nov 2011 - 27 Jan 2012	13	585	Tune gradient boosting settings
Test	3 Feb 2012 - 26 Oct 2012	39	1,755	Final, one-time accuracy measurement
Future	2 Nov 2012 - 25 Jan 2013	13	585	Business forecast (no actuals exist)

Table 4. Time-based data split.

All forecasts for the test period are made *once*, from the end of January 2012, for 1 to 39 weeks ahead, exactly as a planner would forecast the coming months. The test data was not used for any modelling decision.

4.4 Feature engineering

The machine-learning models need the time-series structure turned into columns. Every feature uses only information that would be known at the forecast origin; since the horizon is at most 39 weeks, any lag of 51 weeks or more always points to the past, so there is no leakage.

Feature	Meaning
lag_52	Store's sales in the same week one year earlier (52 weeks; Walmart's Friday weeks keep holidays aligned)
lag_52_smooth	Average of weeks t-51, t-52 and t-53; softens holidays that shift by one week
store_mean	Store's average weekly sales over the training period
store_growth	Store's last 52 weeks of sales divided by the 52 weeks before (year-on-year trend)
lag_52_x_growth	lag_52 x store_growth: last year's sales adjusted for the store's growth
WeekOfYear, Month, IsHoliday	Calendar position and the dataset's holiday flag
Store, Type_code, Size	Store identity, format (A/B/C as 0/1/2) and floor area
Temperature, Fuel_Price, CPI, Unemployment	Regional weather and economic indicators
MarkDown_Total, MarkDown_Recorded	Sum of the five promotional markdowns and the recording flag

Table 5. Features used by the machine-learning models (17 in total).

4.5 Models

Model	Family	How it forecasts
Naive	Baseline	Repeats the last observed week (27 Jan 2012) for all 39 weeks
Seasonal naive	Baseline	Repeats the same week from the previous year (lag_52)
Holt-Winters	Statistical	Additive triple exponential smoothing (level, trend, 52-week season), fitted separately for each store; implemented from scratch
Ridge regression	Machine learning	Linear model with L2 regularisation on standardised features and one-hot store type
Random forest	Machine learning	400 bagged decision trees, min. 2 samples per leaf, 60% of features per split; one global model for all stores
Gradient boosting	Machine learning	HistGradientBoostingRegressor with absolute-error loss, settings chosen by grid search on the validation weeks

Table 6. Models compared.

Holt-Winters updates three components each week, with smoothing weights alpha, beta and gamma between 0 and 1:

$$\text{level: } l_t = \text{alpha}(y_t - s_{t-m}) + (1 - \text{alpha})(l_{t-1} + b_{t-1})$$
$$\text{trend: } b_t = \text{beta}(l_t - l_{t-1}) + (1 - \text{beta})\, b_{t-1}$$
$$\text{season: } s_t = \text{gamma}(y_t - l_t) + (1 - \text{gamma})\, s_{t-m}$$
$$\text{forecast: } y_{t+h} = l_t + h\, b_t + s_{t+h-m}, \quad m = 52$$

The three weights are chosen per store by minimising the squared one-step-ahead errors with SciPy's L-BFGS-B optimiser, started from three different points to avoid poor local minima.

4.6 Hyper-parameter tuning

The gradient boosting model was tuned with a grid of 16 combinations (learning rate 0.03 / 0.1, 300 / 600 trees, 15 / 31 leaves, 10 / 20 minimum samples per leaf). Each combination was trained on Feb - Oct 2011 and scored on the 13 validation weeks (Nov 2011 - Jan 2012). The best setting (learning rate 0.1, 600 trees, 15 leaves, 10 samples per leaf) was then refit on the full training period. The random forest and ridge models use standard settings.

4.7 Evaluation metrics

Four metrics were computed on the 1,755 test store-weeks (lower is better for all):

- **MAE** - mean absolute error in dollars per store-week; easy to explain to a business user.
- **RMSE** - root mean squared error; punishes large misses more heavily.
- **MAPE** - mean absolute percentage error; makes large and small stores comparable.
- **WMAE** - weighted MAE in which holiday weeks count 5 times; the official metric of the Kaggle competition and the main ranking metric here. $\text{WMAE} = \Sigma\, w_i |y_i - y_i| \,/\, \Sigma\, w_i$, with $w = 5$ for holiday weeks and 1 otherwise.

Accuracy was also measured for the **chain total** (sum of the 45 store forecasts per week), which is what head-office planning uses.

5. Code and Implementation

5.1 Project structure

The code is split into small modules with one responsibility each. All files are well commented and the full source is in the `src/` folder of the submission.

File	Lines	Responsibility
main.py	42	Runs the full pipeline (steps 1 to 4) with one command
src/config.py	64	Paths, split dates, season length, holiday weight, colours
src/data_prep.py	133	Loading, quality report, cleaning, aggregation and merging
src/eda.py	205	Six exploratory charts and the EDA summary table
src/features.py	80	Leakage-safe lag, store-level and growth features
src/holt_winters.py	82	Additive Holt-Winters model written from scratch
src/train_evaluate.py	325	All six models, tuning, metrics, error analysis, five charts
src/forecast_future.py	102	Refits the best model and forecasts the next 13 weeks
src/plot_style.py	65	Shared chart style and axis formatters

Table 7. Source files.

How to run: install the requirements with `pip install -r requirements.txt`, then run `python main.py` from the project folder. The whole pipeline finishes in under a minute on a laptop and regenerates every chart and table used in this report.

5.2 Key code

The most important parts of the implementation are shown below, copied directly from the source files.

Data preparation - aggregating departments to store level and merging features (src/data_prep.py)

```
def build_store_weekly(train, features, stores):
    """
    Turn department-level sales into store-level weekly sales and attach the
    external features and store information.

    Why store level?  Store managers plan staffing, inventory and cash flow
    per store per week, so that is the level the business forecasts at.
    """
    # 1) Department -> store aggregation.
    #    Negative department values are product returns; they are genuine
    #    accounting entries, so we keep them inside the store total.
    sales = (
        train.groupby(["Store", "Date"], as_index=False)
        .agg(Weekly_Sales=("Weekly_Sales", "sum"), IsHoliday=("IsHoliday", "max"))
    )

    # 2) Attach economic / weather / promotion features (same store & week).
    #    features.csv has its own IsHoliday column, which duplicates ours.
    df = sales.merge(features.drop(columns="IsHoliday"), on=["Store", "Date"], how="left")

    # 3) Attach store type and size.
    df = df.merge(stores, on="Store", how="left")

    # 4) Missing markdowns: Walmart only started recording promotional
    #    markdowns in Nov 2011, and a blank afterwards means "no markdown that
    #    week". Both cases are filled with 0, and a flag keeps the information.
    df["MarkDown_Recorded"] = df[MARKDOWN_COLS].notna().any(axis=1).astype(int)
    df[MARKDOWN_COLS] = df[MARKDOWN_COLS].fillna(0)
    df["MarkDown_Total"] = df[MARKDOWN_COLS].sum(axis=1)

    # 5) Calendar features.
    df["Year"] = df["Date"].dt.year
    df["Month"] = df["Date"].dt.month
    df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)
    df["IsHoliday"] = df["IsHoliday"].astype(int)

    df = df.sort_values(["Store", "Date"]).reset_index(drop=True)

    # 6) Sanity checks: every store must have every week exactly once.
    n_weeks = df["Date"].nunique()
    assert df.groupby("Store").size().eq(n_weeks).all(), "missing store-weeks"
    assert not df.duplicated(["Store", "Date"]).any(), "duplicate store-weeks"
    assert df[["Temperature", "Fuel_Price", "CPI", "Unemployment"]].notna().all().all()

    return df
```

Feature engineering - seasonal lags and store growth (src/features.py)

```
def add_lag_features(df):
    """
    Seasonal lag features, computed per store:
      lag_52         sales in the same week one year earlier
      lag_52_smooth  average of weeks t-51, t-52, t-53 (smooths out holidays
                     that shift by a week from one year to the next)
    """
    df = df.sort_values(["Store", "Date"]).copy()
    g = df.groupby("Store")["Weekly_Sales"]
    df["lag_52"] = g.shift(SEASON_LENGTH)
    # mean of whichever of the three weeks exist (the very first lagged week
    # has no t-53 value, so it averages t-51 and t-52 only)
    df["lag_52_smooth"] = pd.concat(
        [g.shift(SEASON_LENGTH - 1), g.shift(SEASON_LENGTH), g.shift(SEASON_LENGTH + 1)], axis=1
    ).mean(axis=1, skipna=True)
    df.loc[df["lag_52"].isna(), "lag_52_smooth"] = np.nan
    return df

def add_store_stats(df, history_end):
    """
    Per-store level and growth, measured only on data up to `history_end`:
      store_mean    average weekly sales over the whole history
      store_growth  last 52 weeks' sales / the 52 weeks before that
    lag_52_x_growth = last year's sales scaled by the store's recent growth.
    """
    hist = df[df["Date"] <= history_end].sort_values(["Store", "Date"])

    def yoy_growth(s):
        # compare the most recent k weeks with the SAME k weeks one year
        # earlier (k = 52 when two full years are available)
        v = s.to_numpy()
        k = min(SEASON_LENGTH, len(v) - SEASON_LENGTH)
        return v[-k:].sum() / v[-k - SEASON_LENGTH:-SEASON_LENGTH].sum()

    stats = hist.groupby("Store")["Weekly_Sales"].agg(
        store_mean="mean", store_growth=yoy_growth).reset_index()
    out = df.drop(columns=["store_mean", "store_growth"], errors="ignore").merge(stats, on="Store", how="left")
    out["lag_52_x_growth"] = out["lag_52"] * out["store_growth"]
    return out
```

Holt-Winters from scratch - the smoothing recursions and forecast (src/holt_winters.py)

```
class HoltWintersAdditive:
    def __init__(self, season_length=52):
        self.m = season_length
        self.params_ = None

    # ------------------------------------------------------------------
    def _initial_state(self, y):
        """Initial level, trend and seasonal indices from the first 2 seasons."""
        m = self.m
        season1, season2 = y[:m], y[m:2 * m]
        level = season1.mean()
        trend = (season2.mean() - season1.mean()) / m
        seasonals = season1 - level
        return level, trend, seasonals.copy()

    def _smooth(self, y, alpha, beta, gamma):
        """Run the recursions; return one-step errors and the final state."""
        m = self.m
        level, trend, seasonals = self._initial_state(y)
        errors = np.empty(len(y))
        for t in range(len(y)):
            s = seasonals[t % m]
            prediction = level + trend + s           # one-step-ahead forecast
            errors[t] = y[t] - prediction
            new_level = alpha * (y[t] - s) + (1 - alpha) * (level + trend)
            trend = beta * (new_level - level) + (1 - beta) * trend
            seasonals[t % m] = gamma * (y[t] - new_level) + (1 - gamma) * s
            level = new_level
        return errors, level, trend, seasonals

    # ------------------------------------------------------------------
    def fit(self, y):
        y = np.asarray(y, dtype=float)
        if len(y) < 2 * self.m:
            raise ValueError("Holt-Winters needs at least two full seasons of data")
        self.y_ = y

        def sse(p):
            errors, *_ = self._smooth(y, *p)
            # skip the first season: those errors only reflect initialisation
            return np.sum(errors[self.m:] ** 2)

        best = None
        # a few starting points avoid getting stuck in a poor local minimum
        for start in ([0.3, 0.05, 0.3], [0.1, 0.01, 0.1], [0.6, 0.1, 0.6]):
            res = minimize(sse, start, method="L-BFGS-B", bounds=[(0, 1)] * 3)
            if best is None or res.fun < best.fun:
                best = res
        self.params_ = dict(zip(["alpha", "beta", "gamma"], best.x))
        _, self.level_, self.trend_, self.seasonals_ = self._smooth(y, *best.x)
        return self

    def forecast(self, horizon):
        """Forecast the next `horizon` steps after the end of the training data."""
        n = len(self.y_)
        steps = np.arange(1, horizon + 1)
        season_idx = (n + steps - 1) % self.m
        return self.level_ + steps * self.trend_ + self.seasonals_[season_idx]
```

Evaluation metric - holiday-weighted MAE (src/train_evaluate.py)

```
def wmae(y, y_hat, holiday):
    """Weighted MAE: holiday weeks count HOLIDAY_WEIGHT times more."""
    w = np.where(holiday == 1, HOLIDAY_WEIGHT, 1)
    return np.sum(w * np.abs(y - y_hat)) / np.sum(w)
```

Hyper-parameter tuning with a time-based validation split (src/train_evaluate.py)

```
def tune_gbm(df):
    """
    Small grid search with a TIME-BASED validation split:
      fit on Feb 2011 - Oct 2011, validate on Nov 2011 - Jan 2012
    (the validation weeks include Thanksgiving and Christmas, the hardest
    part of the year). Nothing from the test period is touched.
    """
    origin = pd.Timestamp(VALID_START) - pd.Timedelta(weeks=1)
    feats = build_matrix(df, history_end=origin)
    rows = feats.dropna(subset=["lag_52"])
    fit = rows[rows["Date"] <= origin]
    val = rows[(rows["Date"] >= VALID_START) & (rows["Date"] <= TRAIN_END)]

    grid = {"learning_rate": [0.03, 0.1], "max_iter": [300, 600],
            "max_leaf_nodes": [15, 31], "min_samples_leaf": [10, 20]}
    results = []
    for values in itertools.product(*grid.values()):
        params = dict(zip(grid.keys(), values))
        model = make_gbm(**params).fit(fit[FEATURE_COLS], fit[TARGET])
        score = wmae(val[TARGET].to_numpy(), model.predict(val[FEATURE_COLS]), val["IsHoliday"].to_numpy())
        results.append({**params, "valid_WMAE": score})
    res = pd.DataFrame(results).sort_values("valid_WMAE")
    res.to_csv(RESULTS_DIR / "gbm_tuning_results.csv", index=False)
    best = res.iloc[0].drop("valid_WMAE").to_dict()
    best = {k: (int(v) if k != "learning_rate" else float(v)) for k, v in best.items()}
    print(f"   best gradient boosting params (validation WMAE {res.iloc[0]['valid_WMAE']:,.0f}): {best}")
    return best
```

6. Results and Observations

6.1 Exploratory data analysis

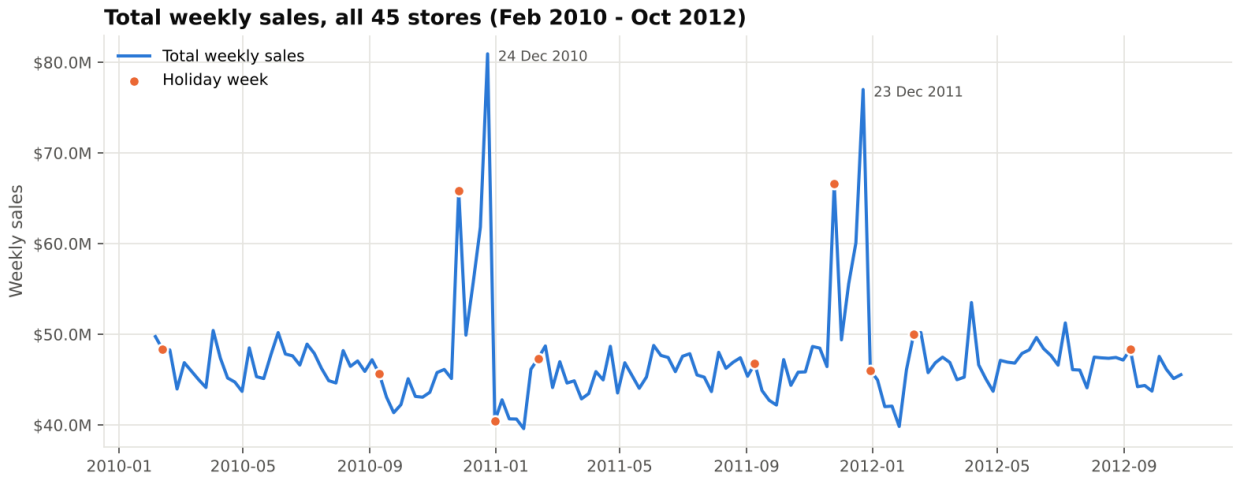


Figure 2. Total weekly sales of all 45 stores. Sales hover around $47.1M a week, with two sharp spikes each year: Thanksgiving week and the week before Christmas.

Observation 1 - a stable level with sharp holiday peaks. Weekly chain sales average $47.1M. The highest week was 2010-12-24 ($80.9M) and the lowest 2011-01-28 ($39.6M): the January slump right after the holidays. Growth over the three years is small (-0.6% from 2010 to 2011 and 2.5% from 2011 to 2012, comparing February-October).

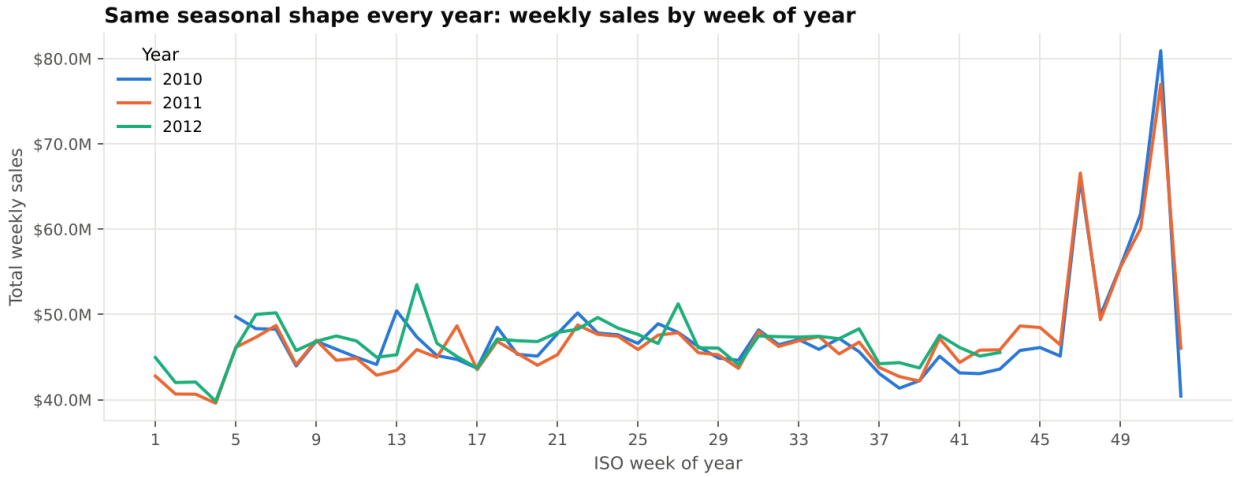


Figure 3. Weekly sales by week of the year: the three years follow almost the same curve.

Observation 2 - strong, repeating yearly seasonality. When the years are overlaid they almost coincide. This is why "same week last year" (seasonal naive) is a strong baseline, and why a 52-week lag is the most useful feature.

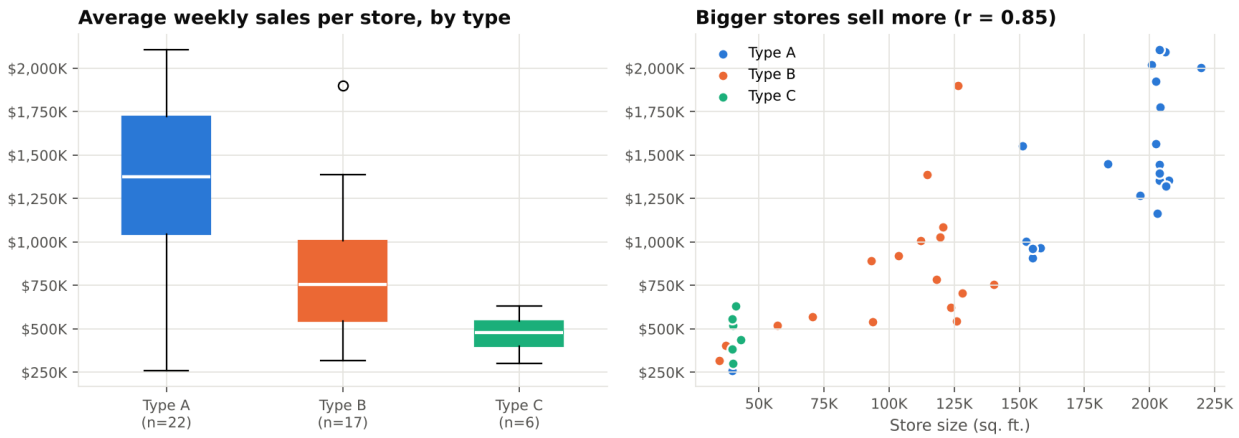


Figure 4. Left: average weekly sales per store by type. Right: store size against average weekly sales (correlation 0.85).

Observation 3 - store size sets the sales level. Type A stores average $1,377K a week, Type B $823K and Type C $473K. Size and average sales are strongly correlated (r = 0.85), so models must handle very different scales; this is why MAPE is reported alongside dollar errors.

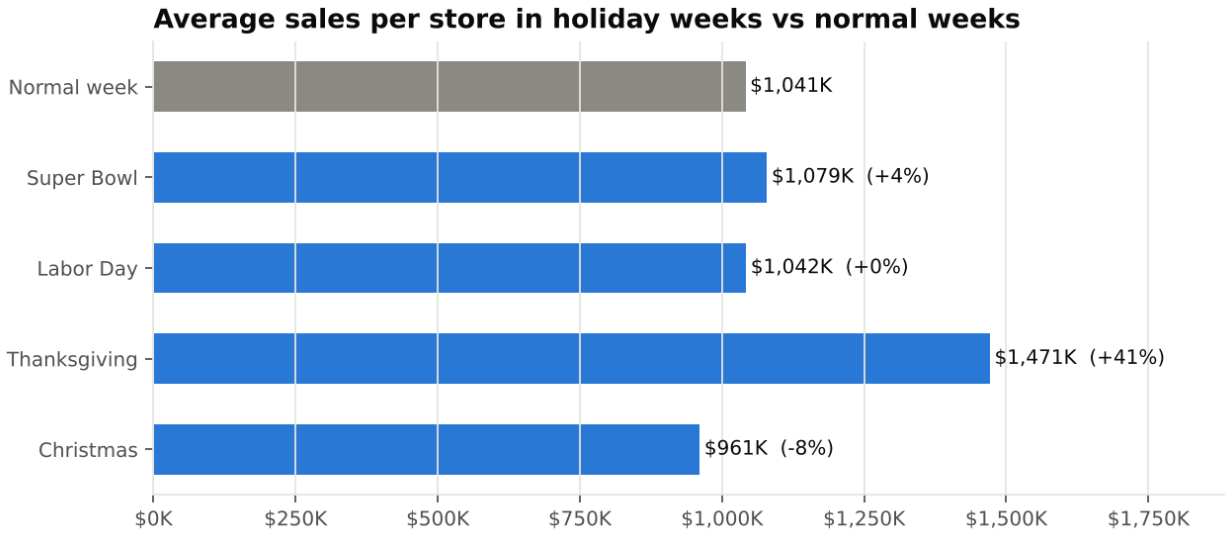


Figure 5. Average sales per store in each kind of holiday week compared with normal weeks.

Observation 4 - not all holidays are equal. Holiday weeks overall are 7.8% above normal weeks, but this hides very different effects: Thanksgiving week is 41.3% higher, the Super Bowl slightly higher, Labor Day unchanged, and the dataset's "Christmas" week (the week after 25 December) is actually *below* normal, because the shopping happens the week before. A single holiday flag therefore explains little; last year's value for the same week captures each holiday's own effect.

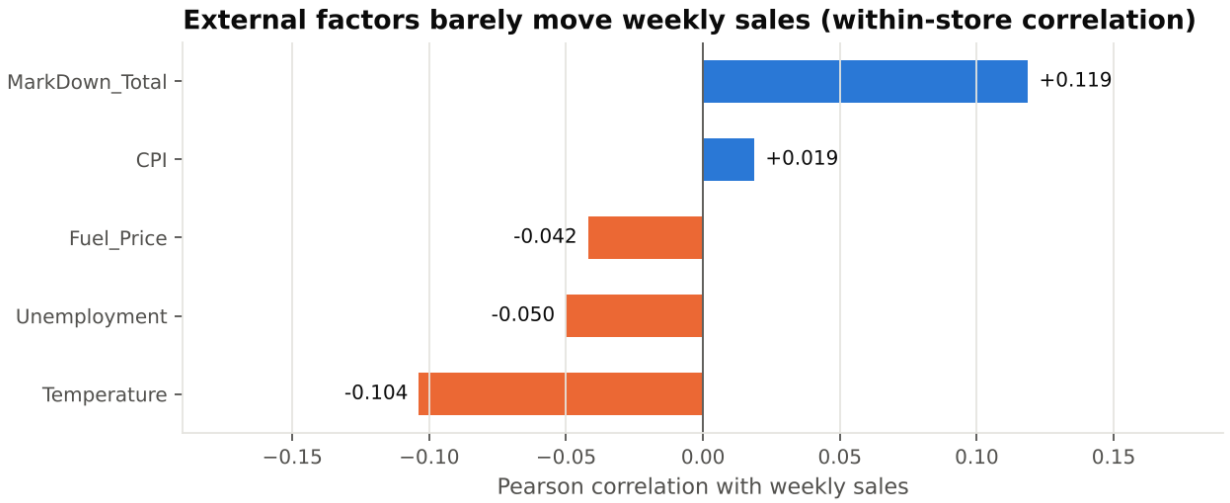


Figure 6. Correlation of each external factor with weekly sales, measured within each store.

Observation 5 - external factors matter very little. After removing differences between stores, temperature, fuel price, CPI and unemployment each have a correlation of 0.1 or less with weekly sales; markdowns are only slightly positive, mainly because they are concentrated in the holiday season. This suggests that calendar and history will be far more useful than economic indicators - which the feature importance results later confirm.

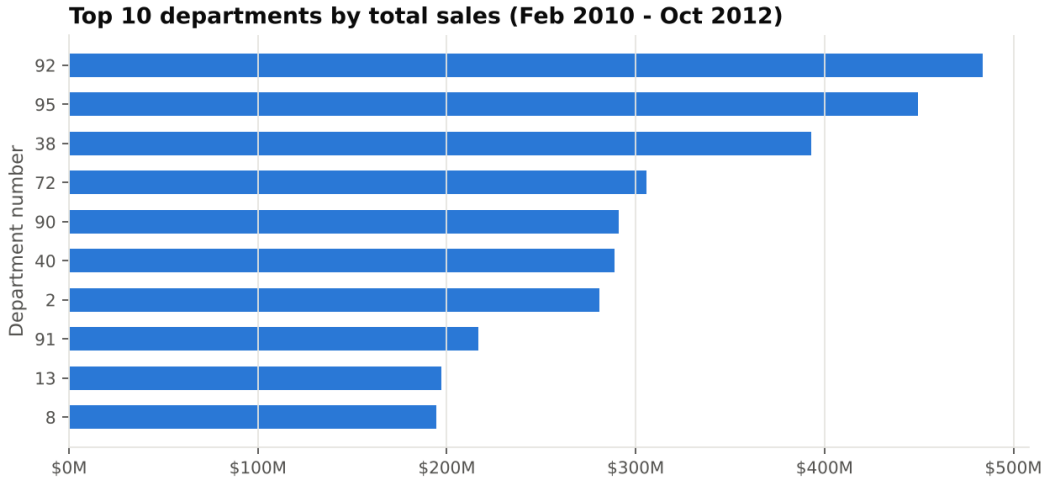


Figure 7. Top 10 departments by total sales; together they make up 46.0% of all sales.

Observation 6 - revenue is concentrated. 10 of the 81 departments generate 46.0% of sales. This project forecasts at store level, but these departments would be the first candidates for a department-level extension.

6.2 Model comparison

Model	MAE	RMSE	MAPE	WMAE	vs. seasonal naive	Chain MAPE
Random forest	$50K	$77K	4.78%	$52K	-12.4%	2.62%
Ridge regression	$53K	$80K	5.41%	$53K	-10.2%	2.96%
Gradient boosting	$56K	$84K	5.39%	$58K	-2.9%	3.11%
Holt-Winters	$58K	$91K	5.86%	$59K	-0.9%	2.60%
Seasonal naive	$58K	$87K	5.90%	$59K	+0.0%	3.14%
Naive	$160K	$206K	14.29%	$168K	+183.9%	15.01%

Table 8. Test-period accuracy of all models (39 weeks x 45 stores = 1,755 forecasts). Sorted by WMAE.

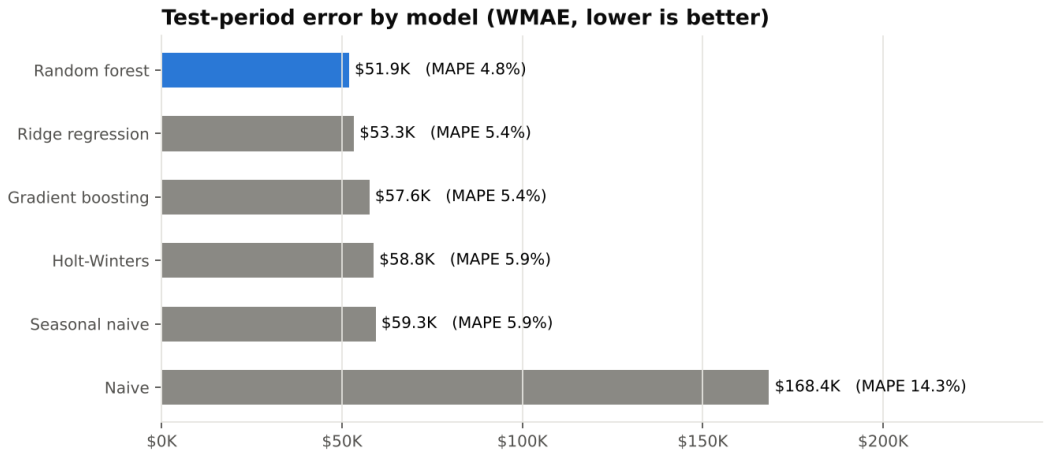


Figure 8. WMAE of each model on the test period, with MAPE in brackets.

Observation 7 - the random forest is the most accurate model. It achieves a WMAE of $52K and MAPE of 4.8%, a **12.4% improvement** over the seasonal naive baseline ($59K) and the best score on every store-level metric. Ridge regression is a close second (-10.2%), which shows that most of the gain comes from the engineered features rather than from model complexity.

Observation 8 - beating seasonal naive is hard. Holt-Winters improves on it by only 0.9%, and the plain naive forecast is almost three times worse ($168K) because it ignores seasonality completely. Holt-Winters has to estimate 52 seasonal values from only two years of data, which limits it; its fitted gamma value was close to 0 for 29 of the 45 stores, meaning it mostly kept its initial seasonal pattern.

Observation 9 - tuning on the holiday season did not transfer. Gradient boosting was tuned on the Nov - Jan validation weeks, which are dominated by holiday spikes, while the test period (Feb - Oct) is mostly ordinary weeks. With only 2,340 training rows it also over-fits more easily than the averaged trees of a random forest, and it finished third (-2.9% vs. seasonal naive). This is a useful lesson: the validation period should look like the period being forecast.

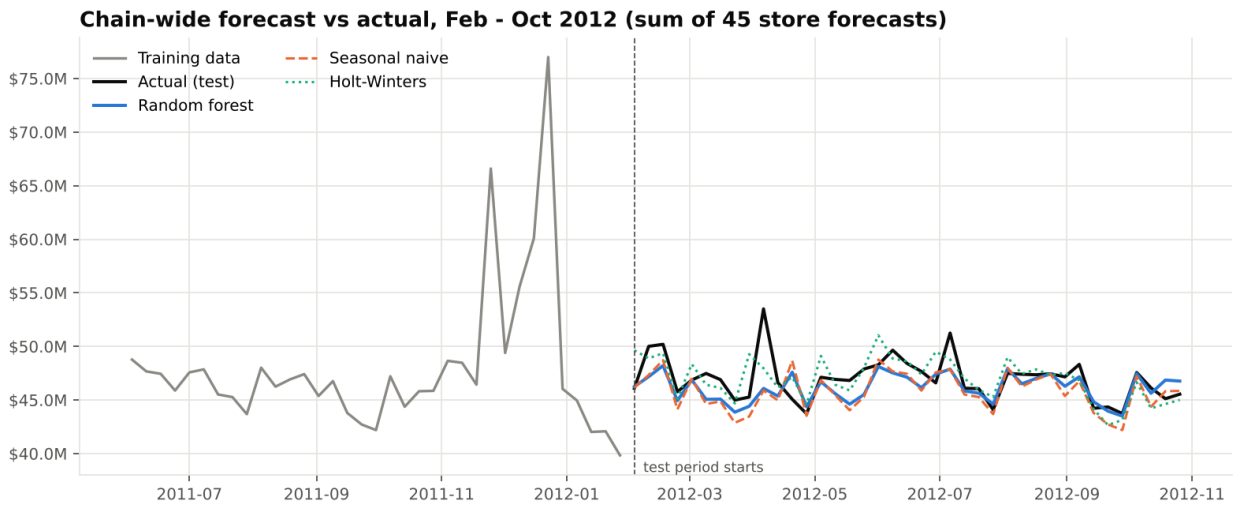


Figure 9. Chain-wide weekly sales: actual values against the forecasts made at the end of January 2012.

Observation 10 - chain-level forecasts are very accurate. When the 45 store forecasts are added up, store-level errors partly cancel out: the random forest is off by 2.6% per week on average and -1.7% on the 39-week total. Holt-Winters is equally good at this level (2.6%), showing that it captures the overall pattern well but is weaker for individual stores. The largest miss is the week of 6 April 2012: Easter fell about two weeks earlier than in 2011 (8 April vs. 24 April), and Easter is not marked as a holiday in the dataset.

6.3 Error analysis

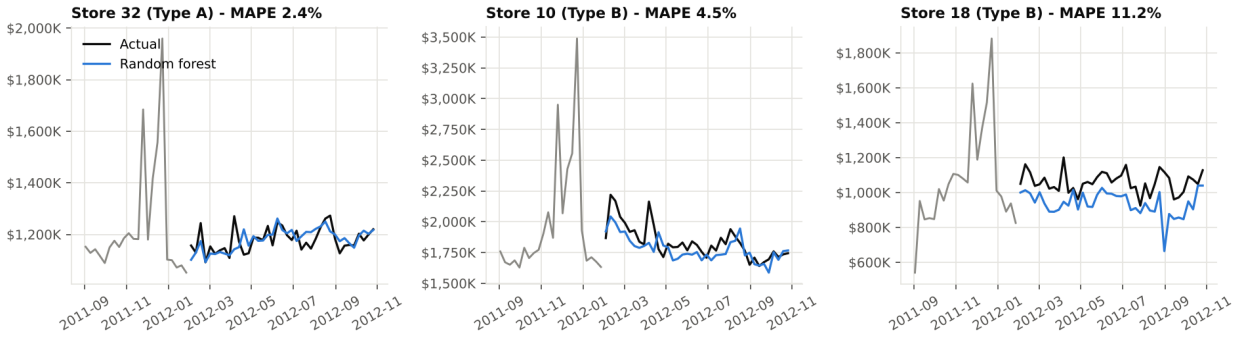


Figure 10. Forecast vs. actual for the best-forecast store, a typical store and the worst-forecast store.

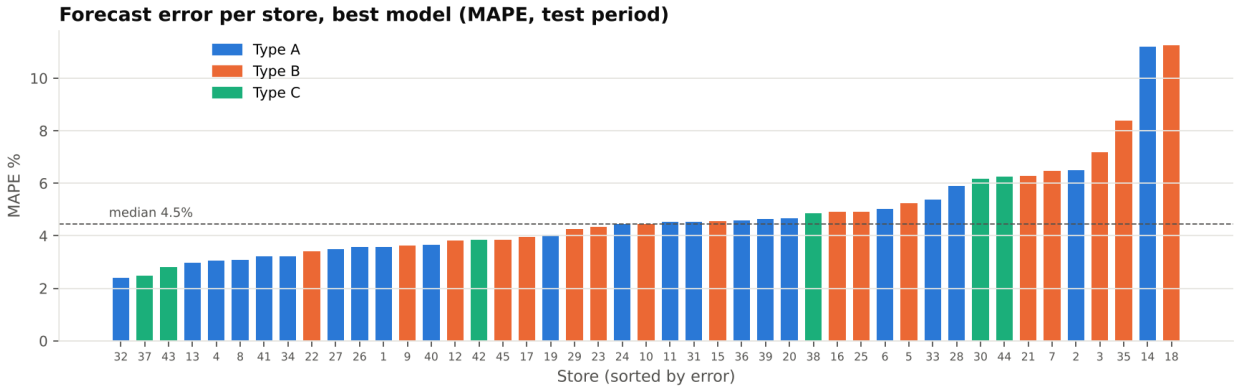


Figure 11. MAPE of the random forest for every store, coloured by store type.

Observation 11 - most stores are forecast well, a few are not. 32 of 45 stores have a MAPE under 5% and the median is 4.5%. Average error is similar for Type A (4.4%) and Type C (4.4%) stores and slightly higher for Type B (5.4%). The two hardest stores, 14 and 18 (about 11% error), changed their sales level during 2012: store 14's sales fell about 9% year on year and store 18's rose about 6%, so last year's level no longer fitted. The forecast for store 18 therefore stays consistently below the actual values (Figure 10), and store 14's above them. Such level shifts are the main weakness of forecasts that lean on last year's data.

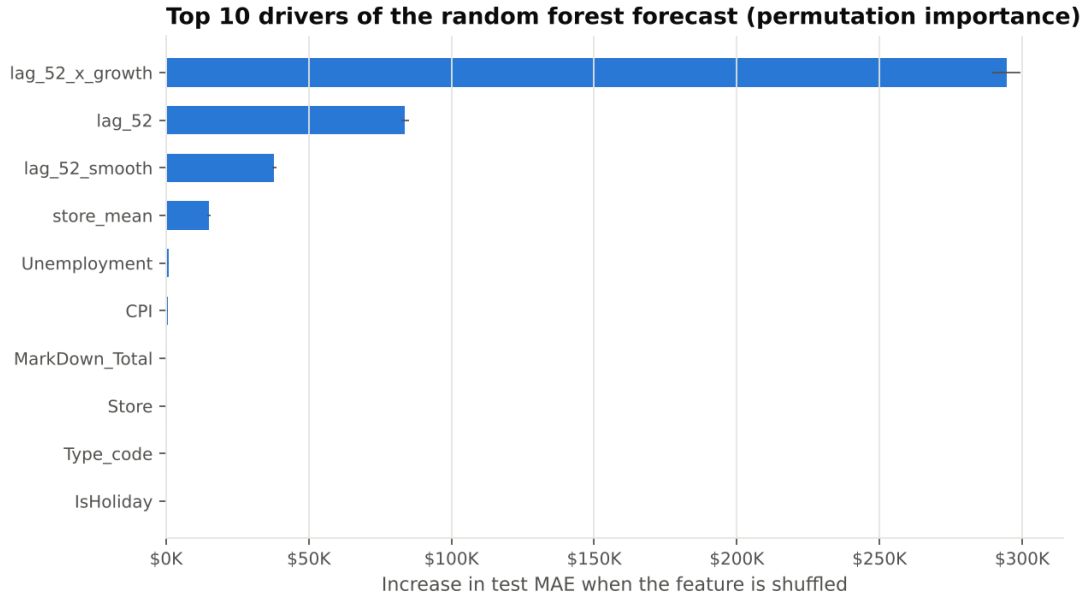


Figure 12. Permutation importance: how much the random forest's test MAE rises when one feature is shuffled.

Observation 12 - history beats external data. The engineered feature *lag_52_x_growth* is by far the most important: shuffling it raises the error by $294K. It is followed by the raw and smoothed 52-week lags and the store's average level. Temperature, fuel price, CPI, unemployment and markdowns add almost nothing, which agrees with the weak correlations found in the EDA (Observation 5).

6.4 Forecast for the next holiday season



Figure 13. Chain-wide forecast for 2 Nov 2012 - 25 Jan 2013 from the random forest refit on all 143 weeks.

Week ending	Forecast	Same week last year	Week ending	Forecast	Same week last year
02 Nov 2012	$49.1M	$48.7M	21 Dec 2012	$74.2M	$77.0M
09 Nov 2012	$48.7M	$48.5M	28 Dec 2012	$48.0M	$46.0M
16 Nov 2012	$47.8M	$46.4M	04 Jan 2013	$46.3M	$45.0M
23 Nov 2012	$65.4M	$66.6M	11 Jan 2013	$43.7M	$42.0M
30 Nov 2012	$50.0M	$49.4M	18 Jan 2013	$43.4M	$42.1M
07 Dec 2012	$55.3M	$55.6M	25 Jan 2013	$41.6M	$39.8M
14 Dec 2012	$59.1M	$60.1M			

Table 9. 13-week chain-wide forecast (store-level values are in outputs/results/future_forecast_by_store.csv).

Observation 13 - the business output. The model forecasts $672.6M of sales over the 13 weeks, +0.8% above the same weeks a year earlier, with the peak in the week ending 21 December 2012 ($74.2M). Every one of the 45 stores has its own 13-week forecast that a store manager could use for staffing and ordering. Note that the peak is forecast slightly below last year's peak: tree models cannot predict values higher than those they were trained on, so extreme record weeks tend to be under-forecast. These weeks have no actual values in the dataset, so this forecast cannot be scored.

7. Conclusion

This project built a complete, reproducible sales forecasting pipeline for 45 retail stores and met each of its objectives:

Objective	Outcome
Understand and clean the data	421,570 rows checked and turned into a gap-free table of 6,435 store-weeks
Find the drivers of sales	Strong yearly seasonality, store size (r = 0.85) and specific holidays drive sales; external economic factors barely matter
Build several model families	Six models: two baselines, Holt-Winters written from scratch, three machine-learning models
Evaluate fairly	Strict time-based split, validation-only tuning, four metrics, store- and chain-level results
Beat a strong baseline	Random forest: 12.4% lower WMAE than seasonal naive, 4.8% MAPE per store, 2.6% for the chain
Explain and apply the model	Feature importance and per-store error analysis; 13-week holiday forecast for every store

Table 10. Objectives and outcomes.

Key learnings

- **Always compare against a simple baseline.** "Same week last year" was hard to beat; without it, a 5.9% MAPE might have looked like a good result on its own.
- **Feature engineering mattered more than the algorithm.** One well-designed feature (last year's sales x store growth) drove most of the improvement, and a simple ridge regression using it came close to the random forest.
- **Time series need time-based validation.** Random splits leak the future; the split, the lag choice and the store statistics were all designed so that no future information is used.
- **The validation period should resemble the forecast period.** Tuning on the holiday season did not help forecasts for spring and summer.
- **Domain detail matters.** The dataset's "Christmas" week falls after Christmas, and Easter moves between years; knowing the calendar explains the biggest errors.

Limitations and future work

- Forecasting at department level (3,300+ series) for more detailed stock planning.
- Adding moving-holiday features (Easter, the exact number of shopping days before Christmas).
- Detecting level shifts such as those in stores 14 and 18, or weighting recent weeks more heavily.
- Testing additional models (SARIMA, Prophet, LightGBM, LSTM) and rolling-origin cross-validation with several forecast start dates.
- Producing prediction intervals (for example with quantile regression) so planners know the range of likely outcomes, not just one number.

8. References, Links and Submitted Files

Dataset and links

- Walmart Recruiting - Store Sales Forecasting (Kaggle competition and data): https://www.kaggle.com/c/walmart-recruiting-store-sales-forecasting/data
- Copy of the data used (public GitHub mirror, folder data/): https://github.com/ShiviBhatt/E-Commerce_Sales_Forecasting
- Project source code: included in the submission ZIP (folder Retail_Sales_Forecasting/).

References

- Hyndman, R.J. and Athanasopoulos, G. *Forecasting: Principles and Practice* (3rd ed.), OTexts - chapters on exponential smoothing and forecast accuracy. https://otexts.com/fpp3/
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. https://scikit-learn.org
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- Winters, P.R. (1960). Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3), 324-342.
- pandas, NumPy, SciPy and Matplotlib documentation: pandas.pydata.org, numpy.org, scipy.org, matplotlib.org

Files in the submission

File	Size
Retail_Sales_Forecasting/README.md	3.0 KB
Retail_Sales_Forecasting/data/processed/department_totals.csv	1.2 KB
Retail_Sales_Forecasting/data/processed/store_weekly.csv	698.8 KB
Retail_Sales_Forecasting/data/raw/features.csv	592.3 KB
Retail_Sales_Forecasting/data/raw/stores.csv	0.5 KB
Retail_Sales_Forecasting/data/raw/train.csv	12.8 MB
Retail_Sales_Forecasting/main.py	1.2 KB
Retail_Sales_Forecasting/outputs/figures/01_chain_weekly_sales.png	116.2 KB
Retail_Sales_Forecasting/outputs/figures/02_yearly_seasonality.png	129.2 KB
Retail_Sales_Forecasting/outputs/figures/03_store_type_and_size.png	104.8 KB
Retail_Sales_Forecasting/outputs/figures/04_holiday_effect.png	66.0 KB
Retail_Sales_Forecasting/outputs/figures/05_external_factor_correlation.png	60.1 KB
Retail_Sales_Forecasting/outputs/figures/06_top_departments.png	44.6 KB
Retail_Sales_Forecasting/outputs/figures/07_model_comparison.png	79.7 KB
Retail_Sales_Forecasting/outputs/figures/08_chain_forecast_vs_actual.png	150.6 KB
Retail_Sales_Forecasting/outputs/figures/09_example_store_forecasts.png	171.2 KB
Retail_Sales_Forecasting/outputs/figures/10_feature_importance.png	74.2 KB
Retail_Sales_Forecasting/outputs/figures/11_per_store_error.png	64.6 KB
Retail_Sales_Forecasting/outputs/figures/12_future_forecast.png	122.7 KB
Retail_Sales_Forecasting/outputs/results/chain_level_error.csv	0.4 KB
Retail_Sales_Forecasting/outputs/results/data_quality_report.csv	0.6 KB
Retail_Sales_Forecasting/outputs/results/eda_summary.csv	0.8 KB
Retail_Sales_Forecasting/outputs/results/feature_importance.csv	0.8 KB
Retail_Sales_Forecasting/outputs/results/future_forecast_by_store.csv	27.7 KB
Retail_Sales_Forecasting/outputs/results/future_forecast_chain_total.csv	0.6 KB
Retail_Sales_Forecasting/outputs/results/gbm_tuning_results.csv	0.6 KB
Retail_Sales_Forecasting/outputs/results/holt_winters_params.csv	1.5 KB
Retail_Sales_Forecasting/outputs/results/model_comparison.csv	0.8 KB
Retail_Sales_Forecasting/outputs/results/per_store_error.csv	3.5 KB
Retail_Sales_Forecasting/outputs/results/test_predictions.csv	217.5 KB
Retail_Sales_Forecasting/requirements.txt	0.1 KB
Retail_Sales_Forecasting/src/config.py	2.5 KB
Retail_Sales_Forecasting/src/data_prep.py	5.2 KB
Retail_Sales_Forecasting/src/eda.py	9.5 KB

File	Size
Retail_Sales_Forecasting/src/features.py	3.1 KB
Retail_Sales_Forecasting/src/forecast_future.py	4.8 KB
Retail_Sales_Forecasting/src/holt_winters.py	3.4 KB
Retail_Sales_Forecasting/src/plot_style.py	2.0 KB
Retail_Sales_Forecasting/src/train_evaluate.py	15.7 KB

Table 11. Every file included in the project folder (the PDF report sits beside it).